

Prototype Overlay Xerographics

(name suggested by R.W.Gosper)

By Robert Elton Maas, with revisions by Ralph E. Gorin and R.E.M.

(POX writeup, edited 1975 DEC 05 06:46 PST, compiled 1975 DEC 05 07:11)

POX is a program for making documents that take advantage of the special features available on the Xerox Graphics Printer (XGP). The special features that both POX and PUB provide are mainly font switching, text justification and automatic "every page" actions such as page numbering. POX also provides vectors, positioning of text etc. to an accuracy of 1/200 inch, composition of text etc. into "overlays" which can be copied and processed without losing the relative positions of the components (somewhat like sets in SUDS, the Helliwell drawing program). POX does not yet provide text responses or footnotes. It is possible to set up cross-references or indexes/contents by means of macros, providing that there are no forward references (i.e. everything is defined before it is used -- POX is one-pass only). Neither POX nor PUB can yet be used interactively like GEOMED and SUDS, and no program yet exists for viewing the final output on displays to check for visual errors prior to XSPooling.

POX has a few intractable bugs -- there is a list of these bugs in a later section of this writeup -- references to these bugs are made from various places in this writeup, in the format "*see bug#n*" where n is the number of the bug in that list -- users should report any bugs they find that are not in that list -- errors in this writeup should also be reported before too many unwary users see them and expect POX to do things it can't do.

POX takes a file as input and produces an output file that is suitable for spooling with the XGP spooler (XSPool command). The input file contains instructions to POX and the text of the document. POX takes character sequences as commands. Each command sequence is preceded by a character, called the *Escape Character*. The first character in the file (ignoring TV directories and SOS line numbers) is taken by POX as the escape character (See note 1). Thus, the user gets to specify the escape character. The escape character can be arbitrary, but reason dictates that it should be some character that appears infrequently (or not at all) in the text of the document. The most popular choices for the escape character are \ _ and \$. In the remainder of this document, the symbol **1** (pronounced "brick" -- you won't find it on your keyboard -- it's ASCII 061 in font 75224X.FNT[1,REM] if you really want to know how I generated it in this writeup) represents your current Escape Character, whatever it may be at the moment; however, in some examples below the actual escape character will be shown rather than the brick symbol.

I haven't really decided what to call these commands, so until make a decision you may find that at some places in this writeup they are called "escape commands" at other places "brick commands" or "POX commands". The commands the XGP understands, beginning with an ASCII 177 which is rubout, have entirely different syntax even for commands that perform almost the same function as a brick command. These are referred to as "XGP escape sequences" or "rubout-style commands" to distinguish them from brick commands. These XGP escape sequences are normally present only in the output from POX and PUB, except when POX is used as post-processor for some other program, such as XLITER. For this and similar use, POX can accept certain system XGP escape sequences at some places in the input file. What POX does is to convert these escape sequences into the equivalent brick command upon input, then to process the brick command in the usual way. Thus the XLITER program can transliterate phonemes into cyrillic characters with rubout-style font selects (to allow direct listing on the XGP if desired), and POX can treat the cyrillic as text, treat the font-selects generated in the process of transliteration as **1**F... commands, and process other brick commands that were in the original text file (that were passed-as-is by XLITER).

To run POX, use the system command R POX (or RU POX [1,REM] if you want to use the latest experimental version of POX if one exists). POX wants to see a command of the form: <OUTPUT FILE><INPUT FILE>. If the <OUTPUT FILE> term is omitted (the arrow must still appear) then the output file will have the same name as the input file, and ".XGP" extension. If only the extension of the output file is omitted, ".XGP" will be used. -- Later if you haven't logged out and have the same alias, you can make POX use the same command, without retyping it, by starting POX at the RPG starting address. For example, you can say "GET SYS:POX" then say "S +1", or from E you can give the extend-mode command "RU POX [1,3]".

If all is well with the input file, POX will run and produce an output file suitable for XSPooling. If you are at a local display, POX will load the appropriate XSPool command into your line editor, so all you have to do is type return to spool the file.

Sample POX input file (I assume you will have your files in E/TV format)

2

```
COMMENT * VALID 00002 PAGES
C REC PAGE DESCRIPTION
C00001 00001
C00002 00002 \|\M0FIX25\M1NGR25\ This is a comment, after the brick-semicolon
C00004 ENDMK
C*;
```

<FORMFEED> i.e. page mark after directory page

```
\|\M0FIX25;\M1NGR25;\; This is a comment, after the brick-semicolon
\; Font #0 is here in case title pages needed -- If we
\; don't explicitly specify it here, the system will invent a font#0.
\<L\+=100;\>L\; Move left margin in by 1/2 inch
\F1\CTITLE OF THIS DOCUMENT IS HERE
```

\J This is a sample document, with justified text, and a centered title. It demonstrates to the person using POX for the first time, just what is involved in a simple POX manuscript (borrowing terminology from the PUB manual, in which a "manuscript" is the uncompiled form you see here and a "document" is the compiled file all ready to output to some device such as lineprinter or XGP). If you have any doubt what the output on the XGP would look like if this document were actually compiled etc., I suggest you create a text file with E or SOS and copy this page (after the formfeed above, until the end-of-file location below), and actually compile it and XSPool it.\.

<END OF FILE>

THIS WRITEUP -- ORGANIZATION, MODES, NOTATION

Each section in this writeup that describes brick-commands consists of the following: Summary of what it's all used for, syntax of each command, the corresponding descriptions of what function each command performs, examples of the use of the commands in this writeup itself with general references as to where to look to see the effect generated, additional examples of use (input text followed immediately by the effect generated in output).

Some brick-commands will put the command scanner in a special mode until such time as an appropriate "get out of this mode" character or brick-command is seen, at which time it will return to its previous mode. Initially (after eating the directory page and the first character or two of the input file) the command scanner is in "PASS-AS-IS" mode which means that carriage-return puts you at the left margin, linefeed moves down to the next line, formfeed completes the page and cuts the paper, brick causes a command to be expected, and all other characters are copied to the output file in the currently-selected font. The effect is that text is aligned against the left margin unless you explicitly do a column-select to cause it to be put elsewhere.

The other modes you can get into are justify, centerify/rightify, copy text into a macro body, evaluate and copy text into an overlay body, evaluate and copy text into a macro argument body, ignore until linefeed, ignore until formfeed, ignore nonprinting characters until semicolon, and parse a brick command. Before each command description is a list of which modes you can be in when the command is scanned. The complete list is (U J CR o A) which refer to Unjustified (normal pass-as-is), Justify, Center/Rightify, atomic text Overlay (1o command) and Argument to a macro respectively (in the other modes, namely define the body of a macro, and parse various commands, the brick character is treated as text rather than initiating a brick-command).

In most of the examples, and in the portion of each syntax specification that is to be typed character-for-character exactly as you see it in this writeup, are in the font KBD25 so that the characters look almost the same as they do on your keyboards and displays (Except that ASCII 000 (null), 011 (tab), 012 (linefeed), 013 (vertical-tab or f), 014 (formfeed), 015 (carriage-return), 040 (space), and 177 (rubout) don't look the same in KBD25 as on your keyboard so that they will be represented in some other way in this writeup). Here is the complete Stanford-ASCII character set in this font:

2

•↓αβ^~επ λγδf±θ∞∂ c∩UVΞ⊕⊕ →~≠≤≥≡v !"#%&'()*+,-./ 01234567 89:;<=>? 3
 @ABCDEFGHIJ KLMNOP QRSTUVW XYZ[\]↑←`abcdefg hijklmno pqrstuvw xyz{|◇}^

All BNF-style syntax is in the font NGR20. Here are the syntactical entities that will be used:

- <TB> -- (horizontal) tab
- <LF> -- line (feed)
- <FF> -- form (feed)
- <CR> -- (carriage) return
- <SP> -- space (bar)
- <CH> -- any single ASCII character (representing data to be copied to output)
- <N> -- any one of these characters: 0 1 2 ... 9 A B ... F (representing a logical font number in IBM hexadecimal notation)
- -- any font file name, if extension and [p,p] are omitted, default .FNT [XGP, SYS] is assumed
- <NAME> -- any alphanumeric string, usually required not to be the null string (an identifier, a name of an overlay, a macro, etc.)
- <OPEN> -- any legal open-nesting character, such as ([{ < , or any legal quoting character, such as " ⅘ ↓ / ∂ |
- <CLOSE> -- the matching close-nesting character, such as)] } > ∩, or the same quoting character, such as " ⅘ ↓ / ∂ |
- <BODY> -- any string of text not containing the quoting character, or containing properly-nested open and close nesting characters (note that all characters are treated as text except for the particular quoting character or nesting pair selected in a particular instance, and special characters like comma etc. which have special syntactical meaning) -- see description in commands where used
- <FILNAM> -- any standard file name, such as SUDS.RPH [UP, DOC].

BASIC COMMANDS --- FONT SELECTION, JUSTIFY, CENTERIFY/RIGHTIFY, IGNORE TEXT

The purpose of these commands is to select the style and size of printing (for example, font BDI25 is BODONI-ITALIC, 25/200 inches high), to enter and exit justify mode (in which the text is massaged so that each line is as full of text as will fit and extra partial-spaces are inserted between words to make the margins line up exactly), to enter and exit rightify mode (in which text is aligned with the right-margin but not modified in any other way), to enter and exit centerify mode (like pass-as-is and rightify except each line is centered), and to ignore portions of the input text that you do not want processed.

FONT SELECTION

- IM<N>; (U) -- causes the font to be loaded into pox (and later into the system XGP service), and selects it now -- see note 5 and bug 2 -- this is the command that sets up the correspondence between fonts and the numbers you will use later to select them -- type to the system HELP FONT to find out about fonts that are available and how to peruse them -- always define one of your fonts as with <N>=0, otherwise the system (which always insists on having font#0 around even if it has to make it up) will waste your time by loading one more font than you are actually using.
- IF<N> (U J CR o) -- selects a font which has already been loaded -- all text after this command before the next IF or IM will be in font#<N>.
- lf<N> (U J CR) -- selects a font for just one character, then back to old font. WARNING -- Do not place the brick character immediately after this command because currently POX will quote the brick character in the temporary font rather than escaping and quoting the first character of generated text in the temporary font -- because this is regarded as a bug it might be changed someday -- currently it generates a warning message.

IGNORE TEXT

- lf (vert. tab) (U J CR o A) -- without changing mode, and without allowing EVERYPAGE to be called here, ignore all input text up to and including the next linefeed, thus concatenation of input lines may be done providing that the line-break is not inside some command syntax -- if you do this at the beginning of a line it is as if the whole line weren't there, thus comment lines may be inserted anywhere in the source file except in the middle of macro-body-definitions and in the middle of the syntax of brick-commands -- this command is especially useful inside

- justify or centerify/rightify mode to create a very long word or a complicated formula or a long sequence of arithmetic operations without leaving the mode. It is also useful in PASS-AS-IS to avoid calling EVERYPAGE just then, and in macros that are likely to be called from almost anywhere.
- IN (U J CR o A) -- same as lf for people who don't like vertical tabs in their text files. (They print out as <LF><LF><LF><LF> on TTY's, dial-up lines and the LPT, as "f" on local DPY's and the XGP, as <erase-to-end-of-screen> on some Beehive terminals, and who-knows-what on other display terminals on the arpanet.)
- 1, (comma) (U J CR o A) -- same as lf except it ignores through next form-feed (page mark) thereby making concatenating two pages into one -- if you do this at the beginning of a page, it is as if the whole page weren't there.
- 1 (space) (U J CR o A) -- without changing mode, ignore all input text (only spaces and tabs allowed) up to and including the next semicolon. This is useful for indenting input-text without generating spaces or tabs in the output. -- Proposed additional feature, if spaces and tabs are terminated by a printing character other than semicolon, that character will terminate this mode but will be unread before returning to previous mode.
- 1 (tab) (U J CR o A) -- same as 1 (space).

ENTER SPECIAL-FORMATTING MODES (J CR)

- 1J (U) -- begins full-justify mode, see table below for details on this mode and how to get out of it. -- Be aware that, because font selects count as text and therefore must be inside words, and because tabs cannot be in the middle of justified lines of text, the following input syntax is illegal and will generate the "illegal tab in justify" error message: <CR><LF>1J1F1<TB>This is a paragraph of text...
- 1R (U) -- begins single-line align-against-right-margin mode, see table below for details on this mode and how to get out of it.
- 1C (U) -- begins single-line center-the-line mode, see table below for details on this mode and how to get out of it.

EXIT SPECIAL-FORMATTING MODES, ETC. (SEE TABLE)

1. (period)
- 1; (semicolon) These two commands will get out of some modes back to pass-as-is mode, and do other things like ignoring text and suppressing <CR><LF> -- see table below for details of what each command does inside each of the modes:

	1.	1;	<CR><LF>	<TB> or <SP>
Pass-as-is	(not supported)	do 1N (see #4)	copy to output (see #4)	copy to output
Justify	finish, do 1N, (see #5)	illegal	(see #1)	(see #2)
Centerify/Rightify	finish, pass-as-is	finish, do 1N, pass-as-is	finish, <CR><LF>, pass-as-is	(see #3)

#1 -- A blank line (i.e. <CR><LF> immediately followed by <CR><LF>) causes the previous paragraph (if any is in progress) to be finished, then a blank line is emitted. As many blank lines as appear in input will appear in output (but see BUG#4). The next nonblank line begins a new paragraph. A single <CR><LF> in a paragraph is merely a break between words.

#2 -- One or more tabs at the beginning of a line causes the previous paragraph (if any is in progress) to be finished, then a new paragraph is started with its first line indented by the tab(s). Tab(s) elsewhere are illegal. NOTE -- There is currently no way to indent by any amount other than a whole number of tabs, nor is there any way to outdent by any amount at all.

#3 -- Tab(s) and/or space(s) at the beginning are ignored (in centerify this makes things look pretty, in rightify it doesn't make any difference). Elsewhere tabs are illegal and spaces are treated as text. (The feature of ignoring tabs and spaces at the end is currently under consideration so don't rely on spaces being treated as text there.)

#4 -- After performing the indicated operation, if the EVERYPAGE feature has been requested for this page (i.e. if VEREVE was TRUE at the end of the preceeding page) but not yet performed, then the body of the macro EVERYPAGE is patched into the input stream at this point. This, of course, is likely to

clobber any registers (including the ac) that are used by the EVERYPAGE macro, unless you carefully write your EVERYPAGE macro so that it saves and restores the ac and any other registers it uses.

#5 -- Register "." is restored to the value it had before you entered justify mode, then control is passed back to PASS-AS-IS mode. If "." was not at the left margin, I make no claims about what POX thinks you are trying to do. I advise you to be sure "." is at left margin before doing 1J in order to avoid trouble and in case the implementation is changed.

Examples of basic commands

In this writeup I loaded the following fonts (\$ is the escape character):
`$M0NGR30;$M1NGR30;$M2BDI25;$M3NGR25;$M4NGB25 [1,REM];$M5NGR20 [1,REM];$M6XMAS25;
 $MFKBD25;$MC75224X.FNT [1,REM];`
 (Note that NGR30 is not really loaded twice because when POX sees the \$M1... command it notices that NGR30 is already loaded as font#0 and POX then merely maps user font number 1 to system font number 0 by means of the font mapping table in POX.

Later in this writeup, in a required-source file, the following load-font commands appear:
`\M0CLAR30;\M1NGB25 [1,REM];\M2NGR20 [1,REM];\M3BDI25;
 \M9GRK30;\MEGRKL50;\MF75224X.FNT [1,REM];`
 Only the fonts not previously loaded are loaded now. The rest are merely mapped to their system font numbers (assigned when first loaded) from their new user font numbers.

This paragraph and all other justified text at normal margins was done by using 1J to enter and 1. to exit justify mode. (See the section on REGISTERS for how indented paragraphs of descriptions after each command syntax was done by the 1j command.) The title on the first page is in centerify mode, using 1C to enter and <CR><LF> to exit. (The titles on other pages are done by overlays, which are also explained later in this writeup.)

Here are some further examples of the above commands:
`$F1This line $F2is in $F3several fonts. $f1Temp $f1font $f1selects
 This line is in several fonts. Temp font selects`

`$F5$CThis line has antidisestablishmentarianismfoogarplylongwordthat is$N
 allinthesinglewordininthemiddleofthislineofcenteredtext as you see.`

This line has antidisestablishmentarianismfoogarplylongwordthat isallinthesinglewordininthemiddleofthislineofcenteredtext as you see.

`$F3Hello there.$RWell, hello to you too$;
 $C(Middle on same line)
 Hello there. (Middle on same line) Well, hello to you too`

`$F3Left part $f1$CThis didn't get centered, see next example for fix
 Left part $CThis didn't get centered, see next example for fix`

`$F3Left part Cf1This got centered ok.
 Left part This got centered ok.`

`$F3For the same reason $f1$'14; doesn't do what you would want.
 For the same reason $'14; doesn't do what you would want.`

`$F3But in this case some commands discussed later, $$F1$'14;$G, do the trick.
 But in this case some commands discussed later, ±, do the trick.`

REGISTERS

There are $2^7=128$ registers which can hold any 36-bit integer (PDP-10 two's complement format), such as the horizontal coordinate of some column on the page. (Note -- all locations on the XGP, whether relative or absolute, are measured in XGP pixels which are about 5 mils, i.e. 1/200 inch) The idea is to allow specification of multi-column tables and *superscripts-above-subscripts* in an easy and flexible way, including automatic allocation of such things as "1/5 of a page" by simple division, and to perform any other integer arithmetic or logic that might be useful during compilations of a document. Each register is specified by an ASCII character. (Note -- in general, POX distinguishes between upper- and lower-case characters -- for example, "k" and "K" do not refer to the same register.) One of the non-alphanumeric registers (currently register V) is called the "accumulator" and

is the intermediate storage for most sequences of arithmetic calculation. Other special registers are listed below, as well as characters that don't specify registers at all. (*Warning* -- non-alphanumeric register names are liable to be changed to special arguments like "." and ";" without much warning. Therefore, please use only alphanumeric register names in files that you will be unwilling to maintain.)

Meaning of argument(s) to the commands:

<CH>	If not mentioned below, contents of register corresponding to that character.
. (period)	Current column location (increases as you move to the right) is stored in the right half of this register. Every time a character is written, this register is updated to account for the space used up by the character, and if you write in this register, a column select will be generated in your output file to really put you there. (U) In pass-as-is mode, this column location is relative to the left edge of the paper and should be in the range of 50 to 1650 (decimal) usually. The left half of this register is zero in this case. (J) In justify mode, it is relative to the beginning of the current word of text (delimited by <SP> or <TB> or <CR><LF> except that <SP> does not delimit if it is not on the baseline). A relocation identifier is in the left half of the register to allow checking for illegal column selects. (CR) In centerify/rightify mode, it is relative to the beginning of the centered/rightified text with a password in the left half as if the text were one big word. -- In atomic-text-overlays and arguments to macros, it is not implemented although you are not prevented from totally screwing yourself by the attempt to store in this register, or getting worthless undocumented data by reading from this register.
, (comma)	This register works somewhat like period, current relative position of text above baseline (if negative, below baseline) -- thus $1\leftarrow\eta 1\rightarrow$, replaces $1\uparrow\eta$ and $1\downarrow\eta$ as well as 1N and 1U . (U J CR)
; (semicolon)	Like "." or "," -- absolute position below top of page (if enabled, see note 4 and the 1c command).
L	Left margin is stored in this register (<CR> causes column select here -- see also 1J 1C 1R 1O 1W).
R	Right margin (for J and CR modes, and for warning you if you try to write past it in pass-as-is mode) is stored in this register.
' <SIGNED OR UNSIGNED OCTAL NUMBER>;	Instead of reading a number from a register, these octal digits are assembled into a constant and that constant acts as an immediate operand (READONLY).
= <SIGNED OR UNSIGNED DECIMAL NUMBER>;	Same as above except decimal conversion (READONLY).
w<CH>	Width of character <CH> in current font (READONLY).
H	Total height of current font (READONLY) (if enabled, see the 1c command).
h	Height above baseline of current font (READONLY) (if enabled, see the 1c command).

The arguments marked "(READONLY)" above are forbidden in certain commands because the command expects to have a genuine register to store data into. This restriction will be indicated by "READONLY operands not allowed" in the command descriptions.

Throughout the remainder of this writeup, command-arguments that that are parsed in the above way will be denoted by various alphanumeric characters in a special font, to distinguish them from characters that you are supposed to actually type (which are in a keyboard-style font as described earlier), and BNF-style syntactical entities (which are in yet another font as also shown earlier). Here are what the alphanumerics look like in the argument font:

abcdefghijklmnopqrstuvwxyz

The commands that use the types of arguments described above:

	LOAD/STORE/ARITHMETIC/LOGIC OPERATIONS
$1\leftarrow\eta$	(U J CR o A) -- load ac from η .
$1\rightarrow\eta$	(U J CR o A) -- store contents of ac into η , READONLY operands not allowed. WARNING -- Do not store in registers "." ";" or ">;" except where legal. POX will get confused if you do. See argument description above.
$1+\eta$ $1-\eta$	
$1*\eta$ $1/\eta$	(U J CR o A) -- usual arithmetic -- ac plus/minus/times/dividedby η ,

7

1% η result in ac.
(U J CR o A) -- "modulo" -- ac divided by η , remainder in ac. -- if the ac is nonnegative and η is positive, this corresponds to reducing the ac modulo η (number theory), otherwise the result is nonstandard mathematically, and indeed machine-dependent.

1- η 1\ η (U J CR o A) -- reverse arithmetic -- η minus/divided by ac, result in ac.
[Note -- since -1 is stored in the PDP-10 as all-ones, the POX command 1-' η -1; will logically complement the ac.]

1^ η 1v η (U J CR o A) -- usual logical operations -- ac and/or η , result in ac.

1L η (U J CR o A) -- logical shift of ac left by η bits (if η <0 then shift right), result in ac.

1~ η (U J CR o A) -- maximum of ac and η , result in ac.

1_ η (U J CR o A) -- minimum of ac and η , result in ac.

SUBSCRIPT/SUPERSCRIP COMMANDS

1 $\uparrow$ η (U J CR o) -- move text up η with respect to the baseline.

1 $\downarrow$ η (U J CR o) -- move text down η with respect to the baseline.

1n (U J CR o) -- up to default superscript location (a fixed portion of the height of current font), easier than storing a positive constant in register η , or setting up a positive constant in some register η and then using 1 $\uparrow$ η .

1U (U J CR o) -- down to default subscript location (a fixed portion of the height of current font), easier than storing a negative number in register η , or setting up a positive constant in some register η and then using 1 $\downarrow$ η .

1 $\otimes$ (U J CR o) -- back to normal baseline from superscript or subscript, don't forget this! -- easier than 1 $\uparrow$ =0; or 1 $\downarrow$ =0; or 1 $\leftarrow$ =0; 1 $\rightarrow$,

UNDERLINE OR FRACTION BAR

1U $\mathbf{latr}$ (U) -- underline from column $\mathbf{l}$ to column $\mathbf{r}$, on scan line $\mathbf{a}$ relative to the baseline ($\mathbf{a}$ >0 moves the underline down), with line thickness $\mathbf{t}$ (extending downward from scan line $\mathbf{a}$ if $\mathbf{t}$ >1). See the example at the end of these commands.

1{ (U J CR o) -- start underline, see UUQ manual under XGP service.

1} $\mathbf{at}$ (U J CR o) -- in conjunction with 1{, end underline, on scan line $\mathbf{a}$ with thickness $\mathbf{t}$ (Note that this command isn't correctly implemented in POX versions before 1975 MAY 11, so don't try to use this feature in old backups of POX or you will get quite confused.)

ENTER SPECIAL-FORMATTING MODES WITH ALTERNATE MARGINS

1j 1r or 1c (U) -- same as 1J 1R or 1C (respectively) except they set up temporary margins from registers $\mathbf{l}$ and $\mathbf{r}$, then restore $\mathbf{L}$ and $\mathbf{R}$ to their old values upon exit. [Note that a <CR> occurring inside one of these commands, such as when <CR> on input terminates 1r or 1c, will set the page printer at the $\mathbf{l}$ rather than $\mathbf{L}$ margin. Thus, although only the $\mathbf{r}$ register needs to be loaded for the rightify per-se, the $\mathbf{l}$ register must be loaded to something reasonable for it to terminate with <CR> correctly.]

GENERATING AND QUOTING SPECIAL CHARACTERS

1'<OCTAL DIGITS>; (U) -- accepts an octal value and transmits it preceded by a rubout. This is used for printing the characters that are hidden under octal values 0,11,12,14,15 and 177. WARNING -- Octal values of 1 thru 4 will cause XGP-escape to occur, then the XGP service will eat one or more bytes of whatever happens to be next in the output file. The result is that if you have the simulator on, or for any other reason cannot predict what the next bytes generated will be, unpredictable things may occur. The only reason POX does not generate an error is because Ralph Gorin insists that old files that do such things as 1'3;\downarrow\downarrow (instead of the preferable 1 $\leftarrow$ '201; 1Y) will continue to work as they always did. (Note, the function of this command is modified by the 1c'1000; or 1c'2000; command.)

1" <CH> (U [J*] A) -- the character <CH> in the current font, preceeded by '177 only if necessary to prevent special things from happening, guaranteed not to go into XGP escape mode. -- When passing character to argument to macro, the character is quoted, i.e. if it is the 1 character or a comma or a nesting/quoting character nothing special happens on account of that -- In justify mode, this command is implemented only for the characters that do not require the '177 prefix 8

11 (U J CR A) [no longer recommended, use 1"1 instead (see previous command)] -- quotes the escape character -- takes precedence over any other command that parses the same due to coincidence of characters, thereby making that other command unavailable -- Please use 1c'1000001; which disables this cruffy old undesirable and unneeded feature, and thus unhides the other command)

1A_q (U [J*] A) -- the seven low-order bits of _q are assembled into an ASCII character, and 1" is performed on that character.

1D_q (U J CR o A) -- converts _q into a string of decimal digits representing its value, like CVS in SAIL.

STACK OPERATIONS

1P (U J CR o A) -- push ac.

1p (U J CR o A) -- pop ac.

1Q_q (U J CR o A) -- push register _q.

1q_q (U J CR o A) -- pop to register _q. -- READONLY operand not allowed.

1# (U J CR o A) -- push the current font number (octal 0 to 17).

1G () -- pop a number off the stack and use that number to do a font-select like 1F... -- must be in the range octal 0 to 17 to work correctly -- not yet fully supported.

TEXT-MODE VECTORS (SEE ALSO ATOMIC VECTOR OVERLAY)

1V_{x₁u₁x₂u₂w} (U) -- draw a vector of **HORIZONTAL** width **w** between (**x₁**,**u₁**) and (**x₂**,**u₂**) -- vectors on the XGP (at SU-A1) are actually tall skinny parallelograms which get increasingly bad-looking as they approach being horizontal -- POX does nothing to improve the looks of diagonal vectors, but exactly horizontal vectors (which wouldn't work at all normally) are replaced by vertical vectors that are very very wide and very very short). Coordinates are measured in usual XGP coordinates. **x** increases from left to right, with **x**=0 being the left edge. **u** increases from top to bottom, with **u**=0 being the top edge. Vectors no longer have to be presented to POX with MIN(**u₁**,**u₂**) in ascending sequence because POX now sorts them into correct sequence automatically, but because the system requires both text and vectors to be sorted correctly, you must usually enable the text linefeed simulator (see note 2) whenever you want ordinary text to appear below any vector or overlay on the same page. One final warning with vectors is not to attempt to do anything at the very top of the first page of output (the system needs time to get started loading buffers).

1v_{x₁u₁x₂u₂t} (U) -- same as 1V_{x₁u₁x₂u₂w} except **t** is thickness perpendicular to the direction of the vector in all cases -- i.e. for non-horizontal vectors $w = \sqrt{t^2((x_1 - x_2)^2 + (u_1 - u_2)^2) / (u_1 - u_2)^2}$, whereas exactly horizontal vectors are fixed up by this command the same as by the 1V... command -- note that vectors which are almost but not exactly horizontal still come out quite ugly and usually overshoot their specified bottom ends.

1w_{x₁u₁x₂u₂t} (not implemented yet) -- even better looking vectors that won't ever overshoot, but because they will be composed of four simple vectors

each they might overload the XGP SER in some cases and not work correctly. 9

REDEFINE THE BRICK CHARACTER

1|<CH> (U J CR o A) -- changes the brick character to <CH> -- I don't guarantee parsing will mesh with this in a reasonable way if you do this deep inside something complicated like justify, surrounded nearby by things that depend on the escape character being what you'd expect -- note that if you do 1|| you can't later change back because then as long as "|" is the escape character, "||" will be parsed as "11" rather than as "1|...". (see the 11 and 1"<CH> commands earlier), unless, of course, you have disabled the 11 command by means of 1c'1000001; or equivalent.

CONTROLLING INTERLINE SPACING

1λ (U) -- use the contents of the ac to set the interline spacing, that is, the minimum number of blank scan lines (1/200 inch) between the very bottom of each line of text (including subscripts) and the very top of the next line (including superscripts). This parameter has different effects depending on whether the simulator is enabled. With the simulator on, if this parameter is negative it allows overlapping, if zero it allows bumping, and if positive it forces a gap, i.e. it is completely logical. With the simulator off it doesn't take effect until after the next text line, and its interpretation depends on its sign: positive is the same as with the simulator on, zero or negative means system default whatever it is today (recently it has had the value 4).

1ε (U) -- use the contents of the ac to set the interbase spacing, i.e. the minimum number of scan lines that baselines are forced apart. When the simulator is on, this parameter together with the interline spacing determine how far apart lines will be (each line is placed as high up on the page as is possible without violating either constraint). When the simulator is off, this parameter is ignored and only the interline spacing has any effect. -- A logical consequence of the above is that if the interline spacing is very negative, then only the interbase spacing has any effect, and you get evenly spaced lines even even if they overlap; whereas if interbase spacing is zero and interline spacing is reasonable, then in most cases only interline spacing will be in control, and you will get lines with the same amount of space between each pair, even if differing amounts of subscripts etc. cause irregular spacing to occur.

RESET VERTICAL POSITION FOR TEXT (SEE ALSO OVERLAYS)

1Y_η (U) -- set absolute Y position for top scan line of this text line relative to top of page (if no text line is currently active, apply to the next text line). Increasing _η moves down the page! Text after this line will be below it on the page unless POX or the XGP SER decides it is too close to the bottom of the page and inserts a page mark. Do not use this command inside your EVERYPAGE macro unless you restore the Y-location of the main page printer before returning from your macro to the mainline text! -- WARNING -- If _η is very large it will probably make the XGP spooler go beserk.

MISC. COMMANDS NOT OFTEN USED

1u_η (U J CR o A) -- generates and executes a CALLI UUO using the right half of _η, thus 1u'400012; will perform a CALLI 400012 -- the ac is used to supply data to some UUO's, and to return data to the program after some UUO'S -- if the ac field of the instruction is used in a special way other than merely pointing to data, then it isn't guaranteed to work the way you want -- the .SKIP. flag (ala SAIL) is returned in register u -- see the UUO manual for details.

1S (U) -- generates a sample of the currently-selected font -- type HELP FONT for other ways to get samples of fonts.

Examples of commands using register arguments etc.

In this writeup, the `lj` command is used to generate the indented paragraph of description alongside the syntax of each command. The sequence `l←nl→.` is used sometimes to set column position for single-line descriptions and to align text in some of the tabular presentations.

Example showing how to do a simple subscript:

```
1F3x1U11⊗+x1U21⊗+...+x1Un1⊗+x1Un+11⊗
x1+x2+...+xn+xn+1
```

Example of making five columns on a page, \$ is the escape char:

```
$←R$-L$/'5;$→W$;                                q-reg W contains width of a column
$←L$→1$+W$→2$+W$→3$+W$→4$+W$→5$;                q-reg's 1,2,3,4,5 contain the column loc's
$←1$→.Hello$←2$→.This$←3$→.Is$←4$→.A$←5$→.Test.
$←1$→.These$←2$→.Columns$←3$→.Are$←4$→.Nicely$←5$→.Aligned.
Hello                This                Is                A                Test.
These                Columns                Are                Nicely                Aligned.
```

Example of hidden characters, \$ is the escape char:

```
$FF$'0;$'11;$'12;$'14;$'15;$'177;
.γδ±⊕^
```

\$F1This shows how to \$←.\$→1underline\$U1=3;=2;. a word in a line of text.
This shows how to underline a word in a line of text.

\$F3\$J This paragraph is an example of how to insert something like X_{i^n} in the middle of a paragraph of justified text using the "." register that represents the current column position relative to the beginning of the current word.
This paragraph is an example of how to insert something like X_{i^n} in the middle of a paragraph of justified text using the "." register that represents the current column ^{loc} position relative to the beginning of the current word.

```
$←=2;$→V$←;$-100;$→A$+=27;$→B$+=45;$→C$;
$←=1200;$→a$+=10;$→b$+=30;$→c$+=30;$→d$+=10;$→e$;
$VaBeBV$VaBdCV$VcAbCV$VcAdCV$VbCeBV$; draw 5-point star of constant width
```

```
$←=2;$→V$←;$-100;$→A$+=27;$→B$+=45;$→C$;
$←=1200;$→a$+=10;$→b$+=30;$→c$+=30;$→d$+=10;$→e$;
$VaBeBV$VaBdCV$VcAbCV$VcAdCV$VbCeBV$; draw 5-point star of constant thickness
```

```
$←;$-20;$→:$v=1400;;=1300;:=20;$; sample of a very ugly vector
```

OVERLAYS

An *overlay* is like a black-on-clear transparency. Overlays can be set up by feeding them text or vectors (someday, video and III/DD buffers also), and several overlays can be placed over one another and glued together to make a compound overlay, which then can be further processed in similar ways. Eventually, overlays are placed in the output file.

All contents of an overlay are positioned relative to an arbitrary *basepoint*. When overlays are placed on top of one-another and glued, the x,y offset of the basepoint of each component is specified relative to the basepoint of the operation as a whole, thus overlays can be made to fit side-by-side or above-and-below or partially-overlapped or whatever. Defaults are available in some of the operations.

In typical applications, overlays act as data, and macros that call the overlay-manipulation commands act as subroutines (see next section for macros).

```
lO<NAME><OPEN><CH><BODY><CLOSE>
```

(U J CR o A) -- [Note that the pair of nesting characters (and) are not allowed in this particular syntax. The reason is that the syntax <OPEN><CH><BODY><CLOSE> will represent something like an atom, with "(" and ")" reserved for generating s-expressions, if and when I implement a subset of LISP inside POX.] if <CH> is "V" then define an atomic vector overlay (syntax is same as the $1V_{x_1u_1x_2u_2w}$ command without the initial 1 character, effect (semantics) is the same except that (0,0) is at the relocatable basepoint rather than at the upper-left corner of the output page) else if <ch> is a period or a font number then define atomic text overlay with font <CH> (the basepoint is set at the intersection of the left edge of the first character with the font baseline) else error -- If there is already an overlay by the same name, the old one may or may not be deleted, so beware.

1n<NAME>; (U J CR o A) -- load parameter n of overlay into ac, see the following table:

n	name	meaning (all quantities are usually nonnegative)
1	LLIM	distance from left edge to basepoint
2	RLIM	distance from basepoint to right edge (LLIM+1+RLIM = total width)
3	TLIM	distance from top edge to basepoint
4	BLIM	distance from basepoint to bottom edge (TLIM+1+BLIM = total height)
5	TLIM1	distance from top nonblank scan line to basepoint (see bug 2)
6	BLIM1	distance from basepoint to bottom nonblank scan line (see bug 2)

1:n<NAME>; (U J CR o A) -- clobber parameter n of overlay from contents of ac, thereby making it "lie" if this parameter is later queried by the 1n command (occasionally useful to interface with macros), only the right halfword is used, interpreted as a signed-halfword in 2's complement notation.

1&<NAME_OF_RESULT><NAME₁>, x_1u_1 &<NAME₂>, x_2u_2 &...&<NAME_n> x_nu_n ; (U J CR o A) -- combine overlays by laying them at various offsets and gluing them together to make a composite overlay -- for i=1,2,...,n the i component overlay is shifted so that its base point is located at (x_i, u_i) relative to the logical basepoint of the resultant overlay -- all six parameters are computed according to the offsets and parameters of the components -- if you leave out the comma and xy arguments from any component, it will place that component just to the right of what is already prepared, and use 0 for the y-offset.

1O<NAME>, xu
 1W<NAME>, xu
 1O<NAME>; (U) -- output overlay to XGP, with basepoint moved to absolute location (x, u) -- you can output an overlay randomly anywhere on the page, or you can replace the " xu " by ";" (see syntax, at left of this paragraph) to get the Gosper-requested default for which no special claim is made, except to say that the overlay is placed "just below" where you were writing plain text previously -- Because text must be presented in ascending Y-sequence to the XGP, you must enable the simulator (see note 2) whenever you place ordinary text below any overlay on the same page. -- ADDENDA 1975.JUL-AUG -- At the insistence of Ralph Gorin, the default way these commands work is a complicated illogical mess that happened to be implemented for a few days in July. If you have the '200000 bit on, as I recommend for all new files, the following more reasonable thing happens: 1O... always affects your page printer, and register ; if the simulator on., and is supposed to be used for displays that occur in the middle of text, not page headers/footings. 1W... never affects your page printer or register ; and is useful for headings/footings, especially from the EVERYPAGE macro feature. The following other differences exist: the 1W... command will force the overlay to be on the same page, even if it is outside the normal page margins, whereas the 1O... command will warn you if it is below the bottom of the page, or if VERPAG is on (see the special features section) it will be placed on the next page automatically.

1@<NEW_NAME><<OLD_NAME>;

12

(U J CR o A) -- rename overlay or macro -- if resultant name is the null string then delete rather than rename -- It is probably a good idea to delete any obsolete overlays or macros to save core -- The only other major reason for using this command now is to temporarily rename something so that it can be temporarily redefined, then delete the temporary definition and rename the permanent definition back again when done -- The rename algorithm currently is as follows: Do all the following, first for overlays, then if that falls thru, for macros: (1) start at the top of list of definitions, i.e. most recently-defined item; (2) if its name equals <OLD> then clobber its retrieval name to <NEW> and return from the Do loop, else if its name equals <NEW> then delete it and fall through to 3, else do nothing and fall through to 3; (3) move down to next older item in the list; (4) go to 2. -- Thus it is possible that renaming the latest version of a twice-defined overlay or macro will unmask the older version. Don't rely on this, it depends too much on the structure of the tables of definitions, and is likely to change in the near future without warning!

Examples of use of overlays

\$oNUM[1-b+SQRT(b\$↑=10;2\$-4ac)]\$; set up the numerator
 \$oDEN[12a]\$←=0;\$→0\$; set up the denominator and a useful register
 \$; NOTE -- The algorithm as presented here only works when NUM is wider than DEN
 \$1NUM;\$→X\$2NUM;\$+X\$→A\$; set up x offset, compute size of numerator
 \$oBAR[V00A0=2;]\$. set up a fraction bar the correct length for numerator
 \$4NUM;\$+=3;\$-0\$→Y\$3DEN;\$+=5;\$→y\$; compute y offsets
 \$1DEN;\$→1\$2DEN;\$-1\$+A\$/=2;\$→x\$; compute x offset
 \$&FRAC<NUM,XY&DEN,xy&BAR,00;\$; make fraction a composite overlay
 \$2FRAC;\$-R\$→X\$4FRAC;\$-; \$→Y\$0FRAC,XY\$; output overlay rightified $\frac{-b+\sqrt{b^2-4ac}}{2a}$

Here's a useful hack using overlays. Suppose you want to find out how wide a string of text is in a particular font (or combination of fonts). You can create an overlay, measure its parameters (1,2,3,4), then delete it right away to reclaim the space used by the overlay that you won't actually be using as an overlay.

MACROS (WITHOUT ARGUMENTS) AND REQUIRE SOURCE FILES

All input to the POX text processor passes through a pre-processor which inserts macro bodies, arguments to macros, and requested source files in the middle of the input stream according to the following commands:

1∞<NAME><OPEN><BODY><CLOSE>

(U J CR o A) -- define a macro for use later -- note that in names of macros, just like almost everywhere else, lower case is not the same as upper case, thus foo Foo f00 F00 f0o fo0 Fo0 and F0o are eight different names -- also, the <BODY> **is not evaluated during this definition phase**, i.e. the escape character and anything else may occur in the body, nothing special happens as the escape character is copied into the macro, escape only occurs later when the macro is actually called, resulting in the escape character and everything else getting stuffed into the input stream.

1!<NAME>; (U J CR o A) -- call a macro, i.e. insert body from definition right here in input stream, hence occurrences of the escape character in the body will cause things to occur now (See note 1 also)

1?<NAME>; (U J CR o A) -- "conditional macro" -- same syntax as above but generate nothing unless accumulator is positive (See note 3)

1@<FILNAM>; (U J CR o A) -- insert source file here then return to current file.

1α<FILNAM>; (U) -- insert source file here but don't come back here -- the rest of this file, and any files that called it, are not processed.

12

1V<FILNAM>; <LIST OF DESIRED MACRO NAMES>;

13

(U J CR o A) -- scan source file that contains macro definitions (see next section on macros with arguments, as well as this section, for details on the macro definitions themselves). At present, the file must contain a TV/E directory to avoid a bug in POX. Nothing in the file is processed unless it simultaneously satisfies these conditions: (1) It begins with a brick character in column 1 of the file, i.e. immediately after a linefeed; (2) It has either a ∞ or 8 in column 2, i.e. it is a command to define a macro; (3) The name of the macro (in the 1∞ or 18 command being tested) is a member of <LIST OF DESIRED MACRO NAMES> that has not yet been used. In particular, 1!... commands are not processed, so that the entire library must use the same brick character, and this brick character must be set up before issuing the 1V... command. Also, 1c etc. commands are not processed, so that all bits needed for correct loading of macros, namely VERREM=40, VERRHT=400 and VERESC=100000, must be in the proper state before issuing the 1V command. For documentation purposes, any library of POX macros should begin with a comment (or a 1c 1> command pair) that reminds users of the correct states of these special-feature flags. -- Example:

1VREMLIB.POX [1, REM] ; DATIME, PAGENUMS, SUB, SUP, PAGENUMS;

This loads the second-occurring version of PAGENUMS on top of (replacing) the first, and loads only the first version of each of the other macros. It is probably preferable to use "fully-cataloged" names rather than have several different macros by the same, then after loading to rename each macro (using the 1d command) to a shorter name that will be used locally. [REM is in the process of compiling a library of some of the generally-useful and requested macros -- 1975 Oct-Dec.]

1β<FILNAM>; (U J CR o A) -- lookup file but don't read any of it. Warn the user if the file does not exist. Do not in any case generate an error or affect the state of the text compilation in progress. Useful for checking all the required source files at the beginning of a long compilation to save time if several files to be called later are missing and need to be restored.

In the above, <FILNAM> is a standard file name FILNAM.EXT [P,P] which must be either a non-POX file with no occurrences of the brick character or a pox file that does not begin with a bare brick character i.e. it should begin with something like 1|1, <OPEN> and <CLOSE> are a matching pair of parenthesis such as () [] { } <> <>, in which case occurrences of these characters in the body must be correctly matched, or two identical quoting characters such as " " // ↓ ↓ ∞ ∞ || ∂ ∂, in which case the quoting character is not allowed to occur in the body at all (the complete current list of legal nesting and quoting characters may be found by examining the source for POX).

<UC> are uncoupling flags (in 1e command, after file name, before ";"), currently unimplemented. Their purpose will be to save various internal states before beginning to input an inserted source file, then to restore those states before returning so that the inserted file won't pass undesired side-effects back to the main program. In the absence of any <UC> flags, any macro-definitions or font compilations etc. done in the inserted source file will stay in effect after returning to the current file, a sometimes undesirable effect.

Examples of macros without arguments

\$∞SUPERSCRIPT [\$F1\$∞\$F2] \$∞NORMAL [\$F1\$∞] \$;
\$!NORMAL; Y = X\$!SUPERSCRIPT; i+1\$!NORMAL; -1
Y = X⁻¹

\$∞SOJGxFOO [\$F2 (FOO) \$←N\$-1; \$→N\$?SOJGxFOO;] \$; set up a count-down-to-zero loop
\$; note that the above macro will recursively call itself N levels deep
\$←15; \$→N\$!SOJGxFOO; \$; call the loop with N=15 to generate 15 copies of (FOO)

(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)

13

MACROS WITH ARGUMENTS -- EVALUATED LIKE LISP EXPR'S 14

Many of the commands described above are too elementary to be less than tedious in practice, even with macros used to abbreviate long sequences of commands. Complicated things are almost unbearable, for example to create a superscript above a subscript as in "x-sub-i-to-the-nth-power" would require quite a mess of almost unreadable code, such as:

```
1F1l⊗x1←.1→01↑=15;1F2n1←.1→11←01→.1↓=12;i1←.1~11→.1⊗1F1
```

where you may have trouble locating the terms "x", "n" and "i" among all the overhead. Using macros without arguments may result in some increase in readability. This can be done in at least two reasonable ways, resulting in:

```
1!txt;x1!savecol;1!sub;i1!swapcol;1!sup;n1!maxcol;
```

or `1⊗main[x]1⊗sub[i]1⊗sup[n]1!sbp;`

Macros with arguments make things easier and much more readable. After once defining the macro `sbp(main,sub,sup)` (see the examples later in this section for details) we can generate x-sub-i-to-the-nth-power by the simple:

```
1!sbp(x,i,n);
```

The pseudo-language we thus build up out of macros-with-arguments partially obviates the necessity of fixing the "bad" syntax that POX has.

DEFINE MACRO -- VERREM SYNTAX

(THIS AND FOLLOWING SECTIONS WERE REWRITTEN 1975.NOV.27-DEC.05)

18<NAME>(<ARGNAMES>)<OPEN><BODY><CLOSE>

(U J CR o A) -- Define a macro with arguments (up to '30 arguments currently allowed). The VERREM special-feature bit must be on at the time this command is scanned. See details of syntax below.

<ARGNAMES> -- Up to '30 alphanumeric strings separated by commas. (Note that the characters surrounding <ARGNAMES> must be parenthesis.)

<BODY> -- An arbitrary string of text to be copied verbatim into the "body" of the macro. This text will be spliced into the input stream later when the macro is called (see the 1!... command), just like macros without arguments. The following exceptions to the "verbatim" copying exist:

<OPEN> and <CLOSE> must be properly-nested if they appear at all inside the body. POX counts how many levels deep you are, and only terminates parsing when the depth reaches zero, i.e. when the <CLOSE> appears that matches the original <OPEN>. Of course, if you use a quoting pair to delimit the body, the quoting characters may not appear at all in the body.

1 (i.e. whatever your brick character is at the time you define the macro) gets converted into a <BRICK> token if the VERESC special-feature bit is on, otherwise it gets copied verbatim too. When the <BRICK> token is seen later when expanding the macro, it gets converted to the 1 character, which will be a different character from the original if you have used the 1|<ch> to redefine the brick character in the meantime. (Without the VERESC feature, it would be the same ASCII character as before, even if it was no longer the brick character.

⊗\$⊗ gets converted into the <BRICK> token regardless of the state of the VERESC flag.

⊗⊗ gets converted into a single ⊗ character.

⊗<ARGNAME>⊗ (one of the argument-names you specified in the <ARGNAMES>) gets converted into the appropriate <ARGn> token. Later when the macro is called, this token is replaced by the nth actual-argument string.

⊗@⊗ (not supported, and likely to be purged or differently-implemented soon) performs the register-argument lookup and uses the value thus obtained (must be in the range 0 to '30) to generate the corresponding <ARGn> token.

⊗...⊗ -- anything else after a ⊗ character is illegal. An error will usually be generated before the second ⊗ is seen (if there is one).

CALLING MACROS -- LIKE LISP EXPR'S

1!<NAME><ARGBATCH><ARGBATCH>...<ARGBATCH>;

where each <ARGBATCH> is of the form <OPEN><ACTUAL ARGS><CLOSE>

and each <ACTUAL ARGS> consists of arguments separated by commas, as described below.

(U J CR o A) -- Call a macro with arguments. Each <ARGBATCH> unit may use a different <OPEN> <CLOSE> pair (when the <OPEN> is seen, POX looks it up in a table to

determine which <CLOSE> to look for) selected from the legal nesting or quoting pairs.¹⁵ As many arguments as desired, separated by commas, may appear in a single <ACTUAL ARGS> item, and as many <ARGBATCH> units may appear altogether (as indicated by the "... in the above syntax specification), providing that the total number of arguments does not exceed '30. The most common way of doing it, and the easiest, is to use ordinary parenthesis, with all arguments in a single <ARGBATCH> unit, for example:

```
1!FOO(First argument,Second argument,Third arg);
```

However, if any argument contains or generates a comma, it is necessary to turn on the VE1LES special-feature bit (to prevent comma from delimiting arguments) and to enclose each argument in a separate set of nesting characters, such as:

```
1!LESHACK(First argument with ",",") (Second argument) (Third);
```

Also, if parenthesis occur inside arguments, it is usually necessary to use some other nesting or quoting pair, for example:

```
1!MESS<This contains (, [, ", and { inside an argument>;
```

A detailed description of the algorithm that is used to scan the macro-with-arguments syntax, build up arguments, bind them to the <ARGn> table that is passed to the macro-body expander, and the algorithm the macro-body expander uses to splice the macro body and the arguments into the input stream, is given later. If you plan to do anything complicated with macros, you will need to read the algorithm-description and understand it in order to predict what happens when various special characters (<COMMA> <OPEN> <CLOSE> and !) are generated inside the scanning-of-arguments process. Note that it is not legal to generate an ASCII '177 or '0 character while preparing an argument, thus many commands (such as font-select) are illegal inside arguments. By quoting the characters that make up the command (see below), however, the evaluation of such commands may be postponed until after the expanded macro has been generated and patched into the input stream, at a time when such commands are legal. Be aware that other commands such as 1A... have a slightly different effect depending on whether they are evaluated while scanning an argument or while generating output text later, and that commands that fetch contents of registers etc. will give different effects if that data changes after argument-scanning-binding time.

```
1?<NAME><OPEN><ACTUAL ARGS><CLOSE><OPEN><A.A><CLOSE>...;
```

(U J CR o A) -- Conditionally call a macro with arguments. At the beginning of the scan, just after the "1?" has been seen, a flag is set according to whether the ac is positive or not. Then all argument scanning etc. is performed just like in the "1!" command (indeed, the same piece of code is executed in POX). When the final semicolon is seen, the flag is tested to determine whether the input-stream patch will be done (if the ac was positive) or the patch not done and the table of arguments flushed (if the ac was zero or negative).

DETAILED DESCRIPTION OF ALGORITHM FOR PROCESSING MACROS

(1) If the command was 1? then the splice-enabling flag is set according to the contents of the ac, but if the command is 1! then the flag is set to TRUE regardless. Text is scanned until a delimiter is seen, and the <NAME> thus scanned is looked up in the table of existing macros. The body of the macro is saved to be used later, and an empty list of arguments is set up. The delimiter at the end of the <NAME>, which should be either a semicolon or an open-nesting or quoting character, is now tested:

(2) If that character was a semicolon, then go to (5); If it was a legal open-nesting or quoting character, then look up the corresponding close-nesting or quoting character in the nesting table inside POX and save it for later reference, then set nesting depth to zero and enter a scanner in which input continues to be read and brick commands continue to be executed as they occur, but output is fed into an argument-body that is being constructed. Paragraph (3) describes this scanning process, and paragraph (4) describes what happens when brick commands are seen during the scanning.

(3) Read a character.

If it is the close-nesting or quoting character then we decrement nesting depth, then depending on its resultant value do the following: DEPTH<0, close off this argument-body, put it in the table of arguments, read one more character, and go to (2) [THIS IS THE ONLY EXIT FROM THIS (3) SCANNING LOOP]; DEPTH=0, do nothing, continue (3) scanning; DEPTH>0, emit the character (close-nesting or quoting) into the argument-body and continue (3) scanning.

Else if it is the open-nesting character then (assuming VERRHT is enabled) increment nesting depth, then depending on the result do the following: DEPTH=1, do nothing, continue (3) scanning;

DEPTH>1, emit the character (open-nesting) into the argument-body, continue (3) scanning. 16

Else if DEPTH>0 then emit the character into the argument-body (thus 1 and <COMMA> are quoted rather than being acted-upon as below) and continue (3) scanning.

Else if the character is a comma then do the following: If VE1LES is on then emit the comma into the argument-body and continue (3) scanning; else close off the argument-body and put it in the table of arguments, then open a new argument-body and continue (3) scanning.

Else if it is the brick character then parse the brick-command as described in (4), and continue (3) scanning when done.

(4) Brick-commands are parsed in the usual way, except that certain commands (such as font-select) are not implemented because they emit rubouts which are not permitted in arguments, and a few other commands that do strange things are also not implemented. When considering brick commands that generate text, it is important to distinguish between those commands which emit a character into the output stream (the argument body) directly (these commands are 1"<CH> 11 and 1A_n) and those which splice text into the input stream (these commands are 1D_n 1!... 1?... 1@... and 1α...). In the latter group of commands, the spliced text will be seen later by the (3) scanner, and if the spliced text contains any of the special characters mentioned in (3), then the corresponding special effect will of course occur. (It is generally a good idea to avoid this except in those cases when such complicated things are actually needed to achieve a complicated effect, as shown in some of the advanced examples below.)

The 1! and 1? commands are especially interesting when they not only are executed while scanning an argument but they contain arguments themselves. What happens is that all status-information about this level of argument-scanning (inside the parsing of the macro-call command that began earlier) is saved on the stack, and this piece of code in POX (i.e. (1) thru (5) as described here) is entered recursively to parse the new level of macro-call and arguments to it. Then when that level of this piece of code returns, after splicing the input stream, that status-information is restored and scanning is resumed at this level as if nothing had happened (except, of course, that since the input stream has been spliced, the output from that lower-level of macro-expansion will now be seen by the scanner at this level).

Users of LISP 1.6 and variants thereof will note, in the case where no parsing side-effects occur (i.e. when <COMMA> <OPEN> and <CLOSE> are not included in the spliced text) and there is exactly one macro call or one quoted string of text in each argument, that the final result of POX macro expansion is exactly the same as the corresponding LISP s-expression EVALuation in which every function is either an EXPR or a MACRO. For example,

1!FOO(1!BAZ(1!FB;);,1!GAR(FG););

is equivalent to the s-expression (FOO (BAZ (FB)) (GAR (QUOTE FG))). In POX there is the additional facility of appending together the strings that are generated by multiple macro-calls and quoted strings in a single argument, for example:

1!FOO(BAZ1!GAR(FG););

is roughly equivalent to (FOO (APPEND (QUOTE BAZ) (GAR (QUOTE FG)))) or something like that.

(5) [The only way to reach this piece of code is to jump here from (2) after seeing a semicolon there.] If the splice-enabling flag was not set above, then all the arguments scanned in (3) etc., and the macro body saved in (1), are now flushed (the only purpose of all this scanning was to make the syntax come out right at this level, although any side-effects (such as storing a value in a register or defining a macro or overlay) during scanning arguments will of course remain done); If, however, the splice-enabling flag was set, then the current input stream is now IOPUSHed so that it can be resumed later when the expanded macro we are generating has become exhausted, and the input is now switched to the macro body we looked up and set aside in (1) above, with the list of argument-bodies standing by to be referenced whenever an <ARGn> token appears in the macro body. In any case, this long-winded piece of code for scanning macro-calls now returns control to whoever called it at this level. (The equivalent thing in LISP 1.6 evaluation of s-expressions is that the call to an MACRO generates an expression which is then re-evaluated by the implicit EVAL that called it while evaluating arguments to a higher EXPR or MACRO -- you may also compare POX macros to EXPR's, which produce the same effect as MACROs, although by a different mechanism -- of course, this parallel breaks down if commas or unmatched nesting characters are returned, in which case rather bizarre parsing side-effects can occur in POX -- in this respect, LISP is cleaner, the only way to make similar things happen is to explicitly mung the stack.) See (6) for what actually happens now when higher-level routines attempt to input characters from the macro-body with argument-list that now constitutes the input stream.

DESCRIPTION OF ALGORITHM FOR READING TEXT FROM A COMPOSITE INPUT STREAM

16

(6) When any routine requests to input a character, the character comes from either an input file or a macro-body or an argument-body (which is treated exactly like a macro-body except that the list of arguments is always null in this case, i.e. arguments cannot themselves have arguments). When any file or macro-body is exhausted, an IOPOP occurs, so that input continues from the next-higher input stream that was suspended earlier. (When the top-level file is exhausted, an END-OF-FILE signal is passed up and causes POX to close out processing and exit.) Finally, if a special token is seen while reading from a macro-body, the following actions are automatically taken: The <BRICK> token is replaced by the current 1 character; The <ARGn> token (if the table of arguments contains at least n arguments) causes an IOPUSH of the macro-body (with associated table of arguments), and causes input to be switched to that nth argument-body, which will now be used until it is exhausted, at which time the original macro-body will resume.

Examples of macros with arguments

Here's the sbp macro definition referred-to earlier, and also used later to demonstrate the argument binding-evaluation process:

```
18sbp(main,sub,sup) [1F11000main01<.1-011=15;1F20sup01<.1-11N
1-01->.1-12;0sub01<.1-11->.1001F1]
```

Here's a macro that combines two overlays into a fraction that is a third overlay, and which works regardless of which overlay is wider:

```
\; *** GIVEN NUM,DEN, RESULT = FRACTION WITH NUM CENTERED OVER DEN AND BAR 3 BITS THICK
\8FRAC(NUM,DEN,RESULT) [\N warning -- registers N,D,B,x,X,y,Y clobbered below...
\10NUM0;\>N\20NUM0;\>N\>N\N compute total size of numerator
\10DEN0;\>D\20DEN0;\>D\>D\N compute total size of denominator
\<N\~D\>B\N compute biggest of two sizes
\<B\>N\>2;\>x\10NUM0;\>x\>x\N compute offset needed for numerator to be centered
\<B\>D\>2;\>X\10DEN0;\>X\>X\N compute offset needed for denominator to be centered
\40NUM0;\>+3;\>-0;\>y\N compute vertical offset to put numerator above frac. bar
\30DEN0;\>+3;\>Y\N compute vertical offset to put denominator below fraction bar
\0BARcV=0;=0;B=0;=3;\>N create a bar of the appropriate horizontal extent
\&0RESULT0<BAR,=0;=-1;\&0NUM0,x\&0DEN0,XY;\N combine the three overlays into one
]\; note that this macro should not be called if the above registers contain data
\; (it might be desirable to use the push and pop commands to save and restore
\; the above-mentioned registers, inside the macro body would be best)
\0FOO [1X\N2\0-Y\N2\0]\0BAZ [1X-Y]\!FRAC(FOO,BAZ,RES);\ORES;\; typical call to the macro
\; Note that "\" must be the escape character at the time FRAC is called or else
\; the "\"s in the body will generate text rather than escape-to-command.
\; This can be remedied by changing each "\" in the body to "\0\" or by turning
\; on the VERESC bit before defining FRAC.
```

Detailed examples of argument evaluation

(The macros called from here, and the fonts used here, are defined in 75224.LMD[1,REM].)

```
\!sbp(X,i-1,n); \;Nothing special here, matching nesting characters, commas
Xi-1n
```

```
\!sbp(X)(i-1)(n); \;One of several other ways to do the same thing
Xi-1n
```

(You must do 1c'400; or equivalent before the syntax of nesting (below) will be allowed.)

```
\!sbp(X,(\f9i)-1,(\f9n)); \;Inside multiple-nested, everything is quoted
\; Thus evaluation of \f9i doesn't occur while binding, but later
Xi-1y
```

\; \!sbp(X,\f9i-1,\f9n); would not have worked, see POINT 7

```
\!sbp(X,(i,j+1),n); \;Even commas are quoted rather than breaking to next arg
Xi,j+1n
```

$\backslash!$ sbp_(n)[X,i,(n)]; \;Nesting only applies to the particular nesting pair in use
X_i

$\backslash!$ sbp_(n)(X,i,((n))); \;Only the pair at comma-level are snarfed
X_i

$\backslash!$ sig; $\backslash!$ sbp(i=1,n); \;Null args are just as good as any others
 $\sum_{i=1}$

$\backslash!$ sbp(NESTING CHARACTERS:.,@//↓↓∂∂|||""^vvUUuu↑↑!!\$%)|() [] {} <> ≤ ≥ ⇔ ⇐|;\;
\;Sometimes you want to pick a nesting character you aren't using in the body
NESTING CHARACTERS:.,@//↓↓∂∂|||""^vvUUuu↑↑!!\$%Z

$\backslash!$ sbp[X,f(i,j),n+2]; \;Avoid expressions like this, their implementation may change
X_{f(i)}

\; -- Define a bunch of month names as if they were elements of an array almost
\∞MONTH1cJanuary>\∞MONTH2cFebruary>\∞MONTH3cMarch>\;
\∞MONTH4cApril>\∞MONTH5cMay>\∞MONTH6cJune>\;
\∞MONTH7cJuly>\∞MONTH8cAugust>\∞MONTH9cSeptember>\;
\∞MONTH10cOctober>\∞MONTH11cNovember>\∞MONTH12cDecember>\;

\8MONTH(NSTR) [\!MONTH@NSTR@;]\; -- Generates the name of month, takes decimal string

\u'400101;\; ACTTIM UUO = CALLI 400101
\L=-18;\/=31;\>D\%=12;\+=1;\>M\; Month number in register M now.
\!MONTH(\DM);\; (1) Scan and evaluate and bind argument, resulting
\; in effectively \!MONTH(4); if this is still April
\; (2) Call macro with argument list to generate a splice
\; in the input stream, which reads: \!MONTH4;
\; (3) Continue reading characters from whatever the input stream is now,
\; namely the string \!MONTH4; followed by the rest of the input file.
\; (4) The \!MONTH4; now binds an empty list of args, calls the macro MONTH4,
\; and splices the string April into the input stream, which is then read
\; and output in the current mode (pass-as-is in this example).

December

LISP programmers will note that this is quite similar to the way EXPRs are evaluated in LISP. In fact, the following macro definition will create a macro that acts like EVAL in LISP, namely EVAL(expression) causes expression to get evaluated twice, once while evaluating and binding arguments, and then again when the result of the macro expansion is seen in the input stream. The usefulness of this is that you can patch together various strings, some quoted (you get what you had in the input file) and some resulting from macro calls or commands like LD, and then the resultant string gets evaluated as if it had been in your input file all along. For example, the above effect of MONTH(NSTR) can be obtained by calling EVAL with the appropriately constructed argument.

\8EVAL(expr) [∞expr@]\; As described above
\!EVAL [\!MONTH(\DM)];\; (1) The following is output to the argument body:
\; (a) The quoted string \!MONTH
\; (b) The decimal string value of register M
\; (c) The close-parenthesis and semicolon
\; (2) The string spliced in is \!MONTH(4); if this is still april
\!EVAL [\!MONTH(\DM)];\; An older way that still works, quote the escape character by \\
\; -- you see it below done both ways on the same line, no difference in effect --

DecemberDecember

\; Evaluate a macro and use the result as a brick command to evaluate:

```
\!EVAL(\!\!FOO;);\; Like \!\!FOO; parsed right-to-left sort-of
```

19

```
$8RECURSExFOO(N)[$F3(FOO)$<=&N&;$--=1;$?RECURSExFOO($DV);]
```

\$; the above macro explicitly calls itself with one less than its argument
 \$!RECURSExFOO{15};\$; {} are just as good as () in such places as here

```
(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)(FOO)
```

Here is a macro that can be used to compile page numbers into a table of contents or index, providing that the contents or index appears in the output file after all referenced items have occurred. This is the macro used in the POX writeup to make the contents and appendix page references.

```
$8DEFCONT(ITEM)[$!EVAL((\&CX&ITEM&){}$<#&--=10;$?SPACEXY;$D#({}));] $;
```

The three types of nesting characters, [] () and {}, are used to isolate nesting processing into three levels in which the other nesting characters are passed as text. This macro assumes that register # always contains the correct page number, and that there is a macro called "SPACEXY" which generates an ASCII 040 (space) character to be used as zero-fill to the left of one-digit numbers, and that page numbers above 99 do not occur. Here's a detailed description of what happens when this macro is used:

\$8.; at start of the file -- A macro with arguments is set up, with the name "DEFCONT", and the body <BRICK> "!EVAL((" <BRICK> "&CX" <ARG 1> {})" <BRICK> "<#&" <BRICK> "--=10;" <BRICK> "?SPACEXY;" <BRICK> "D#({});" where <BRICK> denotes the 1 token (stored internally as 177,175), <ARG 1> denotes the argument-number-1 token (stored internally as 177,001), and other parts are between quote marks here to avoid ambiguity as you try to read this. (This assumes you have VERESC turned on. Otherwise you just get the dollar-sign character in the macro body instead of the 1 token, and what happens later in this example when "\" is the brick character won't work.)

\DEFCONT(A3); at the location in the file desired ("\" happens to be the brick character here) -- The body of the DEFCONT macro is patched into the input stream, and in the process of reading it in the <BRICK> and <ARG 1> are replaced by "1" and "A3" respectively, resulting in the following string being effectively in the input stream: "\!EVAL((\&CXA3{ })\<#&--=10;\?SPACEXY;\D#({}));".

The "EVAL" macro is then called, and in the process of binding its one argument the following happens: (1) The string "\&CXA3{" is passed as (quoted) text; (2) Register # is fetched and tested to see if it is less than ten, if so the SPACEXY macro generates a space character (in the argument-to-EVAL being constructed); (3) Register # is fetched again, converted into decimal string, and appended to the argument; (4) The string "}" (one character) is passed (quoted); (5) The input text ")" terminates the binding of the first argument, and terminates all binding for the call of EVAL, and the body of EVAL, which consists solely of the <ARG 1> token, is patched into the input stream.

Thus if this is done on page 5, the string "\&CXA3{ 5}" (the argument) is now effectively in the input stream. Upon reading all that (generated) input, the macro "CXA3" is defined, with body "5".

Later, for example in the table of contents page, the string 1!CXA3; -- will generate the correct page reference for item A3 in the document, namely the string "5" in this case.

Here is the setup for the EVERYPAGE macro that was used in this writeup (\ is escape char.):

```
\!ACTTIM;\; load a register with system date-time by means of the lu command
```

```
\!EVAL((\&TODAY[ ]\!DATIME;[ ]));\;
```

```
\; call DATIME (in DATIME.POX[1,REM]) and hardwire the result into the TODAY macro  

\&IDENT[POX writeup -- compiled by POX on ]\;
```

```
\&EP1[2\!IDENT;\!TODAY; -- page ]\; hardwire all but the page number into an overlay
```

and here is the actual macro definition (\$ is escape char.):

```
$8EVERYPAGE(dummyarg)[$<#&+=1;$>#&EP2{2 $D#} $&EP<EP1&EP2;$N
```

```
$QA$1EP;$>A$2EP;$>A$/=2;$+=850;$>A$N
```

```
$WEP,A=150;$WEP,A=2100;$qA]$; generate page number, combine overlays, align
```

Note that some tricky things were done in creating the everypage macro so that the time-of-day wouldn't be recompiled on everypage (otherwise if the system were very busy, each page might have a different time-of-compilation. In general, in POX the order of evaluation of arguments may be controlled by means of calls to EVAL (to cause an extra evaluation to occur) and using nesting characters to quote portions of arguments (to suppress or postpone an evaluation). The following

19

example should show you some of the options you can create by use of this technique. 20

Suppose that MONTHNUMBER is a macro that asks the system for the date and time, and decodes the number of month (0 to 11 decimal) leaving it in register M. Suppose that MONTH1 is a macro that is the ASCII name of the first month (January) etc. up to MONTH12 = December. Suppose that MONTHN(n) is a macro with one argument that concatenates the string "\$!MONTH" with its argument (assumed to be a decimal number string) and with a semicolon, so that for example "\$!MONTHN(3);" will patch "\$!MONTH3;" into the input stream which will then generate "March". The following commands will then generate the correct result, assuming this month is neither January nor December:

```
$!MONTHNUMBER;$;
```

```
This month is $!MONTHN($DM); and next month is $!MONTHN($←M$+=1;$DV);.
```

```
Last month was $!MONTHN($←M$-=1;$DV);.
```

Note that if the contents of register M are changed, the result of calling MONTH(\$DM) changes accordingly, and if any of the MONTH1 MONTH2 ... MONTH12 macros are changed (such as to a set of abbreviations for the months) then the result of calling MONTHN will change in the particular cases that were changed. That is because register M is referenced and macros MONTHN(\$DM) and MONTH6 (assuming this is still June) are called every time \$!MONTH(\$DM); appears in the input stream. Thus if THISMONTH is defined as the successive calling of MONTHNUMBER and MONTH(\$DM) then everytime THISMONTH is called register M will be clobbered and MONTHN(\$DM) and MONTH6 will be called. By appropriate control of evaluation, it is possible to call some of those things once at the beginning (when THISMONTH is defined) and hardwire their results into the definition of THISMONTH:

```
$!EVAL(( $∞THISMONTH[] $!MONTHNUMBER; $!MONTHN($DM); [] ) );
```

```
$; "$∞THISMONTH[" is quoted, "$!MONTHNUMBER;" and "$!MONTHN;" are called
```

```
$; resulting in the string "June" if it is still June, "]" is quoted, and
```

```
$; all these parts are concatenated into one string which is passed to
```

```
$; EVAL as its argument "$∞THISMONTH[June]" which is then patched into
```

```
$; the input stream, causing THISMONTH to be permanently defined as "June".
```

```
$!EVAL(( $∞THISMONTH[] $!MONTHNUMBER; ($!MONTHN{} $DM{}; [] ) ) );
```

```
$; "$∞THISMONTH[" is quoted, "$!MONTHNUMBER;" is called resulting in register
```

```
$; M getting defined now, "$!MONTHN{" is quoted, the decimal conversion of
```

```
$; the contents of M we have just computed earlier come next, then "{};]"
```

```
$; is quoted, and all the above is concatenated into
```

```
$; "$∞THISMONTH[$!MONTHN{6};]" which causes THISMONTH to be defined so as
```

```
$; to call MONTHN(6) every time. Thus register M is freed for other use
```

```
$; without affecting THISMONTH but redefinition of MONTH6 will cause
```

```
$; THISMONTH to change accordingly.
```

ENABLING SPECIAL FEATURES

In the following three commands, the left half of η , specifies a batch of features, and the bits in the right half specify which features in that batch are affected (or in the case of the first command, which are on, since all are affected one way or the other in this obsolete command). Thus you may change as many features in one batch at a time, but must use separate commands for each batch. The commands are given first, then the descriptions of each feature, with the batch number and bit position (mask) that correspond to it. All bits are initially zero, and bits not mentioned below should be left that way to allow for new bits that will be defined later.

1` η (U) -- Set special features halfword from right half of η -- This command is not recommended because it clobbers bits you may have set earlier and forgotten.

1< η (U) -- Use right half of η . as a mask for turning on bits, leaving the rest unaffected.

1> η (U) -- Use right half of η . as a mask for turning off bits (a "1" in the mask turns the bit off, the same as the ANDCM instruction), leaving the rest unaffected.

Before you use any of the above commands to change these flags, all the bits listed below are not-enabled (zero bit). Generally it is advisable to turn these bits on at the beginning and leave them on, however it is frequently useful to toggle them on and off. If funny things can happen with toggling, I will try to warn you in the description below -- please excuse the sloppy (unjustified) appearance, the text was copied from the source program for POX with very little editing. also, the "\$" character

rather than "l" represents the brick character below.

21

I recommend that all newly-created POX files turn on VERSMT, VERXX1 and VEIQES near the start of the file. Also desirable most of the time are VERSIM, VERREM, VERRHT, VERESC and VEIOPN. All of these can be turned on by: 1c'0302444;1c'1000003; at the very beginning of your source file. I recommend against using VER56P, VERREG and VERIDC.
; THESE BITS ARE IMPLEMENTED CURRENTLY:

Batch (left half of η) = 0

VERHHR<-000001 ;ENABLE H AND h HEIGHT OF FONT PSEUDO-REGISTER -- Has its effect at the moment you attempt using any command that takes a register as an argument, and you specify register "H" or "h".
VERTRC<-000002 ;TRACE AND OTHER TTY OUTPUT
VERSIM<-000004 ;ENABLE <LF> SIMULATOR, ENABLE SIDE EFFECT OF REGISTER ";"
;This simulates XGPSE enough to know at all times where it is on the page
;It allows sorting each output thing according to Y0 so that XGPSE doesn't complain
WARNING -- This bit can not be turned off at all (I haven't implemented it because it doesn't seem like a good idea for several reasons) -- Regarding turning it on, if you turn it on after one or more pages of text have been output, the bottom margin will probably get screwed up -- If you turn it on after text has been output on the current page, the xspooler will probably abort due to incorrect Y0 sequence -- In conclusion, you should turn on the simulator before any text is output at all, or not at all.

WARNING -- You must do at least one 1F... after turning on the simulator before outputting any text, or you will get screwed.

;000000 000010 ;(RESERVED FOR FUTURE SGK-JUSTIFIER)

VER56P<-000020 ;ENABLE \$5 AND \$6 INITIALIZATION WITH TLIM1 AND BLIM1 DATA --

See BUG#2 -- Takes effect when a \$M command is executed on a font not previously loaded.

VERREM<-000040 ;ENABLE REM'S SYNTAX FOR MACROS WITH ARGS DEFINITIONS -- Takes effect when the macro is being defined.

VEREVE<-000100 ;ENABLE EVERYPAGE MACRO

;If on, once on every new page of output it simulates 1!EVERYPAGE; occurring in your input file (at the first occurrence of <CR><LF> or 1;...<CR><LF> while at pass-as-is mode, or a few other places -- If you want this to happen on the page where you turn on the feature, you will have to do 1!EVERYPAGE; yourself once there.

WARNING!! -- Because of the fact that this macro is called at an almost-random location with respect to your input file, it must be constructed so as to avoid clobbering anything (registers or input syntax, usually) that might confuse the main processing of the file. Any registers used should be saved and restored using push/pop commands. Watch out for \$f \$N \$; \$, etc. which will cause strange and confusing destruction of your world if the terminating <crlf> <ff> etc. is missing from your everypage macro.

;000000 000200 ;(RESERVED FOR FUTURE COLIST-KLUDGE)

VERRHT<-000400 ;ENABLE MODIFIED WAY TO PARSE ARGS THAT HAVE NESTING CHARS IN THEM (i.e. anything inside nesting inside nesting is quoted, etc., see the earlier section "Detailed examples of argument evaluation")

Takes effect when a parameter is being parsed prior to being bound to dummy arg.

VERREG<-001000 ;\$'<OCTAL>; OLD WAY -- 177 ALWAYS EVEN IF IT DEFEATS POX QUIETLY

VERSMT<-002000 ;\$'<OCTAL>; NEW WAY -- 177 ONLY IF NEEDED

VERIDC<-004000 ;SPECIAL REQUEST BY RHT, NO ERROR IF ; OMITTED AFTER MACRO CALL

VERPAG<-010000 ;AUTOMATIC PAGE CHANGE W/OUT FIRST WARNING, TEXT AND \$0 OUTPUT OVL

VERBG1<-020000 ;ENABLES BUG#1 WARNING MESSAGE (TEMPORARY)

VERLCS<-040000 ;CHANGE WAY TABS IN PASSIS ARE DONE TO 1/10 OF RMAR-LMAR * INPUT COL

WARNING -- Only tabs in input file do the right thing. A counter keeps track of what column in the input file we are at, and rounds up to multiple of 8 when a tab occurs in the input FILE (not in a macro expansion patch). Thus any text just after a tab in the input file will appear on the output page in approximately the same column on the XGP. Note that a tab in a macro will activate this mechanism, but it will go to a position corresponding

21

to the input FILE rather than what that file would have been with the macros inserted in it. Note that the three low order bits of the column counter are ignored also. A warning message will be generated if a tab is encountered in a macro while not at a multiple of 8 columns, until such time as an implementation is found that satisfies everyone.

VERESC←100000 ;CAUSES BRICK CHAR IN MACRO BODY TO BE SAME AS \$\$\$ FEATURE, i.e. the <BRICK> token rather than the character itself -- This bit is tested at the time the macro body is being parsed and associated with the macro name (at define-time during execution of the 18 or 19 command).

WARNING!! (75 AUG 03) -- The token in the macro body is replaced by the current brick character at the time the token is rescanned as input, not earlier when the macro body is patched into the input stream, so if your brick character at macro-define time was \ and the macro body includes \|\$...\$| it will be compiled as <brick>|\$...\$|<brick> then if when you call it \ is the brick character, the <brick>|\$ will be converted to \|\$ and \$ will be your brick character for ..., then \$|<brick> will be converted to \$|\$ instead of the \$| you probably intended, and you will totally lose just like this POX writeup did before I located this subtle bug (in this writeup) and fixed it. If you want the effect desired (whatever the brick char is, change it to \$, then back again) you can probably make it work by having your macro call EVAL with a quoted string containing the brick character, thereby forcing both <brick> tokens to be replaced by the same brick character at argument-binding (arg to EVAL) time and used in both instances. Note that this subtle problem happens the same with the old \$\$\$ syntax in macro bodies.

VERXX1←200000 ;IMPLEMENTS CORRECT OUTPUT-OVERLAY AS OF 1975.JUL/AUG

VERVER←400000 ;ALLOWS TYPEOUT OF SPECIAL FEATURES ON/OFF

Batch (left half of q) = 1

VE1QES←000001 ;SUPPRESSES CRUFTY OLD \$\$\$ "FEATURE"

VE1OPN←000002 ;OUTPUT PAGE NUMBERS TYPED

For example, the command 1c'446; will turn on the simulator and enable the full REM macro-with-argument syntax with ability to quote portions of arguments and cause trace output to appear on your terminal as POX runs. The command 1c'1000001; will turn on the bit that deletes the 11 command. At a later time 1c'2; will turn off the trace without affecting the simulator or macro-with-arguments or any other special-feature bits.

REMARKS ON USING THE POX PROGRAM

If you have the VERTRC flag enabled (see the 1c command et.al.) various random things will get typed out while POX is running, including text in your input file appearing after 1; and 1, commands. For the most part you may ignore them. If you really want to know about them, ask REM about particular ones, or look in the source program for POX. These trace outputs are all subject to purge at any time as POX develops.

To enable some of the new features implemented after 1974 Nov which conflict with old features, special flags must be set earlier in your input file (see the 1c command et.al.) -- The definition of register V as the accumulator, and the newly-implemented side-effects of q-register ", " are automatically in effect and can not be inhibited except by using an old version of POX (before 74.DEC).

Backtrace package

There are 5 kinds of errors that can occur while POX is compiling a text file: (1) E40 = bugs in the user file from which it might be possible to continue if the parser is correctly synchronized or if it will re-synchronize itself soon enough to be able to do the rest of the file mostly correctly. (2) E40F = "fatal" bugs in the user file from which continuation is not possible but context information is available and might be helpful to the user (3) ERRF = errors caused by the input file that have wiped out all context information -- This usually happens at the end of a file while inside the mode of building up the body of a macro or macro-argument, caused by a missing parenthesis somewhere. (4) FATAL = bugs in POX in code that is in a state of debugging and still has ASCII warning messages for detected errors (5) INTERR = internal errors detected in POX program, in code that shouldn't have anything wrong with it, probably core clobberage rather than POX bugs.

In E40 errors the error message is typed out, then the current level of macro expansion or file

input is typed out in about the same way as the SAIL compiler announces compilation errors. When the user has the following options:

- Hit P to proceed with the program;
 - Hit <ctrl><meta>P to clobber the error-call to a NO-OP and proceed. Thus if the same error happens no message will be generated and it will proceed as if you had typed P each time;
 - Hit R to remember the current location in the source file and proceed;
 - Hit <ctrl><meta>R to remember, clobber, and proceed;
 - Hit T to pop up one macro level and do a context display there too [the pop-up is for the displayed output only, it doesn't change the point where the program would resume if P were hit later]
- If you have popped up to an actual input (DSK) file, or were there already, you can't pop up further because implementing that capability in an obvious way WOULD clobber the context;
- Hit E to return to display editor at this location in the input file. The only way that currently exists for getting from E back to POX automatically with the correct top-level file, is to have a do file that says: R POX<<FILE>> and to type to the system DO <DO FILE NAME><CR><LF> the first time you run POX, then afterward to use the <CTRL><META>X G<CR> command in E;
 - Hit <ctrl><meta>E to return to display editor at the location you had it remember earlier;
 - Hit D to go into debug mode (i.e. RAID). Hit <ctrl>P to return from RAID to the backtrace package;
 - Hit <CALL> to do anything else, like to edit with SOS, or to give up entirely, or to "F DSK" then see what POX outputted before the error occurred.

From E40F errors all the above are available except those which involve proceeding.

If you want to examine your output file according to how the XGP service parses it, R XGPTYP. For investigating new bugs in a large file, a useful technique is to run XGPTYP on both the output from a backup working version of your file and the output from the version you are debugging, then R SRCCOM and TTY:<<OLD XGPTYP LISTING>, <NEW XGPTYP LISTING>.

If your file looks ok but you get the message XGP SER MISSED IT. then try appending to the end of the XSPool command the switch /NTNODE=31, then if it still doesn't work discuss it with RWG or REG who are experts at this.

Known Bugs in POX (Referred-to earlier in writeup)

BUG#1 -- [Fixed 1975 MAY/JUN]

BUG#2 -- Font FORN25 and other fonts with funny width characters cause a fatal error, and parameters 5,6 don't usually work correctly. [new bug since 15 and 16 were implemented, the bad code has now been patched-around, so that normally no bug will occur but 15 and 16 will give incorrect data, but if you use the 1c'20; switch you take your chances -- someday the code will be really fixed.]

BUG#3 -- After some errors that happen in justify mode, typing P will hang in a loop in which an erroneous error is repeatedly hit -- someday I might figure out what is happening here.

BUG#4 -- In justify mode, N blank lines in input generate 2N-1 blank lines in output. I haven't figured out a simple way to fix it, so beware.

NOTES (Referred-to earlier in writeup)

(NOTE 1) -- Everywhere, except at the start of the top-level file, POX reads characters in the normal sequence, and expects all syntax to be left-to-right decodeable. This is why, for example, the string 11!FOO;, if seen in top-level left context, is parsed as [quote the escape character, then output the string "1!FOO"], rather than as [expand the macro FOO then execute the command that results from a brick character in front of it].

However, at the start of the first top-level input file a special initialization parser is employed that pre-reads the first two characters then decides what to do regarding syntax. The following table shows the first characters and what happens (here "V" means any character not mentioned previously in the same context, "VV" means the two copies of some character, "V3" means two different characters):

C -- assumes this is a TV/E directory -- skips everything until ASCII '014 (Form Feed) is seen, then prereads again using only the options below this one.

V| -- assumes this is a "change the escape character" command -- jumps into the middle of the 1|<CH> code -- in my opinion, this is the preferred way to begin a POX file and all new files should

begin this way -- for example, your file might begin with `\|\M0KBD25;\M1BDR30;\;` which would define `\` as the 1 character, change it to `\` (i.e. no change), define two fonts, and suppress the `<CR><LF>` at the end of the line -- Note that because most users of POX prefer to use `"\"` as their brick character, if you plan to use some character other than `\` as the brick character in your file, and you want your file to be callable (with minimum hassle) as a required source file from some other file, you should probably begin your file with `\|1` (where 1 denotes your local brick character) and put `1\` at the very end of the file. -- Note that this does not apply to libraries of macros, in which the `1|` command is currently ignored, and you should avoid including it (thus all macro definitions will use the same brick character throughout your file).

`VV` -- assume this is a bare (out-of-syntax) occurrence of the escape character for purpose of defining it the old way (1973 MAR-APR), followed by the beginning of a POX command -- jumps into the middle of the escape code at the point just after the 1 has been recognized.

`V.` -- assume this is an old-style `1;` command (don't rely on this, it is incorrect) -- ignore through next `<crlf>` then jump into beginning of PASS-AS-IS loop.

`V;` -- assume this is a skip-to-crlf command -- does it (see above)

`V3` -- assumes the first is the escape character and that the user is confused about what to do next -- jumps into the PASS-AS-IS loop just after a character has been read, to be able to act upon the `3` character -- because `3` is not the brick character, an error will be generated because POX will attempt to output the `3` character but no font has yet been defined.

(NOTE 2) -- THIS NOTE PURGED -- SEE COMMANDS `1` 1< 1>`

(NOTE 3) -- Of course, since the expansion of a macro generates a new string of text as if it had been in the input file originally, syntax can be altered if the body of a macro begins but does not complete some escape sequence. Thus the definition `1=FOO[1oBIGMESS{4]` will set it up so that `1!FOO;` will begin the definition of an atomic text overlay but not finish it, until some time later in the file an unmatched `"}` occurs. This can be confusing, so watch your macros carefully.

(NOTE 4) -- In normal mode, letting the system do `<LF>` itself or simulating what it does, subscripts etc. cause irregular spacing of lines because each line takes up just as much space as it needs to avoid overlap. If the system is doing `<LF>`'s all by itself (the only mode currently implemented), the POX program hasn't the vagueest idea where on the page the current line of text will be located when the system gets around to putting the output onto the XGP. However, if POX is simulating what the system would do, then POX is using register `";` and a couple of internal locations to keep track of the last scan-line used, etc.

In pretty-spacing mode, the baselines of the text lines are caused to be evenly-spaced even if overlap occurs or font size varies. This is done by computing where the top scan line should be (Y position of baseline desired, minus Y offset of worst-case superscript) and telling the system to do the line there (BLETCH! BLETCH!). This mode is obtained by setting the `€` parameter to correct line spacing, and setting the `λ` parameter sufficiently negative to allow all overlap that is expected without spreading lines further apart.

(NOTE 5) -- If the font file is already compiled, the `1M` command merely maps from the logical font number you selected to the already-compiled system font number. If the font you requested is new but the logical font number is already in use, it finds a slot, loads the font, and maps the font number.

TRIVIA

PASSI4 -- Tell user, at PASSI4+3 is a CAI (NO-OP) that is executed only when POX tries to output a semicolon at passis-level. Patching it to a HALT or setting a breakpoint there is useful for debugging a mysterious extra semicolon that you've created. If that doesn't find it, try patching `PUTCHR+1` too.

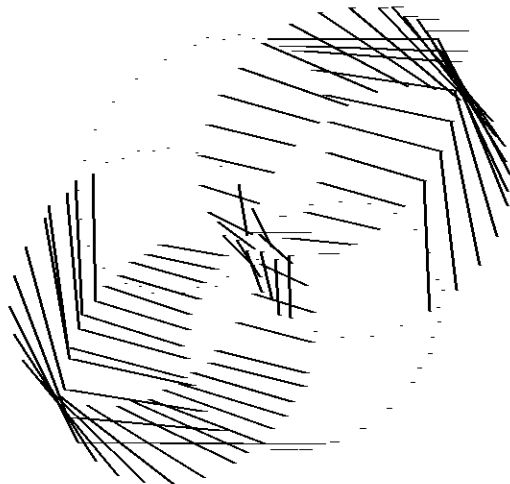
As long as there is room in your mind, I might as well mention some major deficiencies of POX. There is no way to place overlays in justified or center/rightified text. There is no way to do automatic footnotes or indexes or cross-references. Syntax is mostly strange.

ONE FINAL TOUCH TO THIS WRITEUP -- The spirograph near the bottom of this page (or the next if there wasn't room here) is a simple example of how to use recursive macros to make program loops. It also shows you the overshoot that happens with almost-horizional vectors of the `1v` type.


```

_ ; DRAW SPIROGRAPH STARTING AT (X,Y)/D (x,y)/d offset by (8,9) 25
_∞SPIROc_←X_→a_←Y_→b_←x_/d_+y_→y_+Y_→Y_←y_/d_→x_→x_+X_→X_ ; wheel #1 perturbs #0
_←X_/D_+Y_→Y_←Y_/D_→X_→X_ ; wheel #0
_←X_+8_→A_←a_+8_→a_←Y_+9_→B_←b_+9_→b_vabAB=2;▷_ ; and offset for XGP
_ ;
_∞SPIROSc_!SPIRO;_←I_-=1;_→I_?SPIROS;▷_ ;
_ ;
_←=800;_→8_←=1750;_→9_ ;
_←=-44;_→X_←=26;_→Y_←=2;_→D_ ;
_←=-29;_→x_←=69;_→y_←=1;_→d_ ;
_←=200;_→I_!SPIROS;_ ;

```



APPENDIX -- SUMMARY OF ALL POX COMMANDS

26

IN THE APPROXIMATE CHRONOLOGICAL ORDER IN WHICH THEY WERE IMPLEMENTED

page#

```

↓      [Original commands by R.E.Maas 1973]
;0      ALL OTHERS, ILLEGAL
9      ;1 |      SET NEW ESCAPE
3      ;2 M      MAKE A NEW FONT
4      ;3 ;      IGNORE THRU LF ETC.
4      ;4 ,      IGNORE THRU FF
3      ;5 F      FONT SELECT
3      ;6 f      FONT SELECT FOR ONE CHARACTER
9      ;7 S      SAMPLE
4      ;10 J     JUSTIFY
7      ;11 j     JUSTIFY
4      ;12 C     CENTER
7      ;13 c     CENTER
4      ;14 R     RIGHTIFY
7      ;15 r     RIGHTIFY
6      ;16 ←+*\/\~^v%L~ Q-REGISTER ARITHMETIC
6      ;17 →     Q REGISTER DEPOSIT
7*     ;62 ↑↓    Q-REGISTER SUB/SUP-SCRIPT [GENERATES ESCAPE SEQ. VIA XCT W]
7      ;20 nu⊗   MOVE BASELINE.

      [Added by R.E.Gorin 1973-74]
7      ;21 U     UNDERSCORE
8      ;22 P     PUSH
8      ;23 p     POP
7      ;24 '     QUOTE WITH RUBOUT <<NOT RECOMMENDED>>
9      ;25 λ     SET SPECIAL LINE SPACE FROM AC0
8      ;26 V     VECTOR
9      ;27 Y     Set Y position

      [Added by R.E.Maas 1974-75]
12,14 ;30 ∞     Define macro
12,14 ;31 !     Call macro
12,15 ;32 ?     If QREGAC>0 then call macro else scan but null semantics
12    ;33 @     Require source file, then return
12    ;34 α     Jump to file, don't return
8*    ;26 v     pretty vector
8      ;35 Q     PUSH WITH ARG
8      ;36 q     POP WITH ARG
10    ;37 o     DEFINE ATOMIC TEXT OVERLAY
11    ;40 O     OUTPUT OVERLAY
11    ;41 123456 LOAD QREGAC WITH PARAMETER FROM OVERLAY <<5,6 NOT WORKING>>
20    ;42 `     [TEMPORARY] SET UP VERSION FLAGS <<NOT RECOMMENDED>>
11    ;43 &     COMBINE SEVERAL OVERLAYS INTO ONE, WITH OFFSETS EACH
12    ;44 ∂     RENAME OVERLAY IN COMPLICATED WAY
11    ;45 :     Clobber selected size parameter of overlay
9      ;46 €     SET MINIMUM BASELINE SPACING, USED IN CONJUNCTION WITH λ
4      ;47 <SP><TB> IGNORE SPACES UNTIL ; SEEN (FOR PRETTY SOURCE FILES)
3      ;50 <VT>N BREAK TO NEXT LINE HERE WITHOUT AFFECTING SYNTAX
9      ;51 u     CALLI UUD
7      ;52 "     QUOTE THE NEXT CHARACTER (WITHOUT RUBOUT UNLESS NEEDED)
          NO PAGE CHANGE NEXT TIME <<NOT SUPPORTED>>
8      ;54 A     CVT QREG TO CHARACTER (WITHOUT RUBOUT UNLESS NEEDED) AND OUTPUT
20    ;55 c     TURN ON DEBUG FLAGS -- VLEVE←VLEVEL∨ARG
20    ;56 ∃     TURN OFF DEBUG FLAGS -- VLEVEL←VLEVEL∧(¬ARG)
7      ;57 {     START UNDERLINE [WRITES ESCAPE SEQUENCE VIA XCT W]
7      ;60 }     STOP UNDERLINE [DITTO]

```

26

```

8      ;61 D      CONVERT DECIMAL TO STRING MACRO AND SPLICE NOW [INPDL]
11*    ;40 W      SAME AS O BUT FORCES ON SAME PAGE, DOES NOT AFFECT PAGE PRINTER
13     ;63 β      DO LOOKUP ON FILE, WARN IF MISSING, BUT DON'T READ IT
13*    ;33 V      SCAN OF LIBRARY OF MACRO DEFINITIONS
8      ;64 #      PUSH FONT# ON STACK
8      ;65 G      POP FONT# FROM STACK AND SELECT

```

[Not yet implemented -- 1975.Nov]

```

;N1 X      (RESERVED FOR EXTENDED-SCANNER TO BE IMPLEMENTED SOMEDAY)
;N2 π      (RESERVED FOR ILLEGAL COMMAND TO ENTER DEBUG PACKAGE)

```

INDEX TO POX COMMANDS IN ASCII-ALPHABETICAL ORDER

NUMBER REFERS TO SUMMARY OF COMMANDS IN PREVIOUS TABLE

AN ASTERISK HERE MEANS IT'S NOT LOCATED IN NUMERICAL ORDER THERE

	xx0	xx1	xx2	xx3	xx4	xx5	xx6	xx7
00x	NUL 00	↓ 62*	α 34	β 63	^ 16	¬ 16	€ 46	π N2
01x	λ 25	TB 47	LF 00	f 50	FF 00	CR 00	∞ 30	ð 44
02x	c 55	↳ 56	n 20	u 20	V (64)	∃	⊗ 20	↔
03x	_ 16	→ 17	~ 16	≠	≤	≥	≡	√ 16
04x	SP 47	! 31	" 52	#	\$	% 16	& 43	´ 24
05x	((?)	)	* 16	+ 16	, 04	- 16	. 03	/ 16
06x	Ø	1 41	2 41	3 41	4 41	5 (41)	6 (41)	7 (?)
07x	8 30	9	: 45	; 03	<	=	>	? 32
10x	@ 33	A 54	B	C 12	D 61	E	F 05	G
11x	H	I	J 10	K	L 16	M 02	N 50	O 40
12x	P 22	Q 35	R 14	S 07	T	U 21	V 26	W 40*
13x	X N1	Y 27	Z	[	\ 16	]	↑ 62*	← 16
14x	` 42	a	b	c 13	d	e	f 06	g
15x	h	i	j 11	k	l	m	n 53	o 37
16x	p 23	q 36	r 15	s	t	u 51	v 26*	w (26)
17x	x	y	z	{ 57	01	ALT 00	} 60	BS 00

(To use the above table to locate the command corresponding to a particular character, look up the character according to ASCII number, use the numerical entry in the table to reference the preceeding table, and read the description on that table. That first table now includes page numbers for your convenience in locating the full description elsewhere in this writeup. The table of contents, however, will usually be more useful for locating general types of commands.)

PROTOTYPE OVERLAY XEROGRAPHICS -- WRITEUP

CONTENTS

(page numbers here correspond to headings/footings -- add one for XSPPOOL command page numbers) ↓↓

Contents [this page, generated after p.27, bound before p.1]	<i>i</i>
Introduction	1
Sample POX manuscript	2
This writeup, organization, modes, notation	2
BNF notation used in this writeup	3
Basic commands	3
Table of modes and commands	4
Examples	5
Registers and XY coordinates	5
Arguments to commands	6
Commands (most of the POX commands are in this section...)	6
...Vectors	8
...Interline spacing	9
Examples	10
Overlays	10
Examples	12
Macros without arguments, require source files	12
Examples	13
Macros with arguments	14
Commands	14
Detailed algorithm of evaluation	15
Examples	17
Detailed examples of argument evaluation	17
Advanced examples (MONTH<N>, EVAL, RECURSE, DEFCONT, EVERYPAGE)	18
Special-feature bits	20
Batch 0	21
Batch 1	22
Examples	22
Remarks on using the POX program	22
Backtrace/Error package	22
Known bugs in POX (by number)	23
Notes (by number)	23
Trivia	24
Appendix	26
Chronological list of commands (with page references)	26
ASCII sequence table	27

i